



**Universidad Politécnica
de Chiapas**

UNIVERSIDAD POLITÉCNICA DE CHIAPAS

Ingeniería en Tecnologías de la Información y la Innovación
Digital

Examen: Diseño de sistemas y Base de Datos
“DOCUMENTO DE DISEÑO: SISTEMA DE LOGÍSTICA DE
DISTRIBUCIÓN REGIONAL”

Integrantes:

Espinoza Mendoza Héctor Isaac - 243

Moreno Zuñiga Jesel - 243

Arturo Yion Jaime - 243602

Docente: Ramsés Camas Nájera

Materia: Bases de Datos Avanzadas

SUCHIAPA, CHIAPAS, 05 FEBRERO DE 2026

ÍNDICE

Examen: Diseño de sistemas y Base de Datos	1
“DOCUMENTO DE DISEÑO: SISTEMA DE LOGÍSTICA DE DISTRIBUCIÓN REGIONAL”	1
ÍNDICE	2
1. INTRODUCCIÓN Y ANÁLISIS DEL CASO	4
1.1 Contexto del Negocio	4
1.2 Problemática Identificada	4
1.3 Objetivos del Diseño	4
2. MODELO CONCEPTUAL (ER) CON JUSTIFICACIONES	5
2.1 Entidades Principales Identificadas	5
PROVEEDOR	5
CATEGORÍA	5
PRODUCTO	5
ENTRADA_ALMACEN	6
DETALLE_ENTRADA (Relación N:N)	6
ZONA_GEOGRAFICA	6
COMERCIO	7
PEDIDO	7
DETALLE_PEDIDO (Relación N:N)	7
VEHICULO	8
CONDUCTOR	8
RUTA	8
PEDIDO_RUTA (Relación N:N)	9
ENTREGA (Entidad débil)	9
3. ESQUEMA RELACIONAL CON JUSTIFICACIONES	11
3.1 Transformación del Modelo ER a Relacional	11
Reglas aplicadas:	11
3.2 Tablas Resultantes	11
Tabla 1: CATEGORIA	11
Tabla 2: PROVEEDOR	12
Tabla 3: PRODUCTO	12
Tabla 4: ENTRADA_ALMACEN	13
Tabla 5: DETALLE_ENTRADA	14
Tabla 6: ZONA_GEOGRAFICA	15
Tabla 7: COMERCIO	15
Tabla 8: PEDIDO	16
Tabla 9: DETALLE_PEDIDO	16
Tabla 10: VEHICULO	17
Tabla 11: CONDUCTOR	17

Tabla 12: RUTA	18
Tabla 13: PEDIDO_RUTA	19
Tabla 14: ENTREGA	19
3.3 Resumen del Esquema	20
4. RESTRICCIONES IMPLEMENTADAS	21
4.1 Tipos de Restricciones	21
4.1.1 Restricciones de Unicidad (UNIQUE)	21
RESTRICCIÓN 1: RFC único	21
RESTRICCIÓN 2: Placas únicas	21
RESTRICCIÓN 3: Licencia única	21
RESTRICCIÓN 4: Nombre de categoría único	21
RESTRICCIÓN 5: Nombre de zona único	22
4.1.2 Restricciones de Validación (CHECK)	22
RESTRICCIÓN 6: Formato de RFC	22
RESTRICCIÓN 7: Precios positivos	22
RESTRICCIÓN 8: Stock no negativo	22
RESTRICCIÓN 9: Cantidades positivas en detalles	23
RESTRICCIÓN 10: Estados válidos para PEDIDO	23
RESTRICCIÓN 11: Estados válidos para VEHICULO	23
RESTRICCIÓN 12: Capacidad positiva	24
RESTRICCIÓN 13: Estados válidos para RUTA	24
RESTRICCIÓN 14: Unidades de medida válidas	24
4.1.3 Restricciones de Obligatoriedad (NOT NULL)	25
4.1.4 Restricciones de Integridad Referencial (FOREIGN KEY)	25
Con DELETE CASCADE:	25
Sin CASCADE (protección):	26
4.2 Restricciones Adicionales (Triggers necesarios)	26
RESTRICCIÓN 15: Peso de ruta no excede capacidad de vehículo	26
RESTRICCIÓN 16: Vehículo disponible para asignación	27
RESTRICCIÓN 17: Actualización automática de stock	28
4.3 Resumen de Restricciones	30
5. VIEWS	31
6. Queries SQL de Ejemplo	33
7. Preguntas	33

1. INTRODUCCIÓN Y ANÁLISIS DEL CASO

1.1 Contexto del Negocio

La distribuidora opera en un entorno multi-regional (Chiapas, Oaxaca, Tabasco) donde debe coordinar tres procesos críticos:

1. **Gestión de Proveedores y Almacén:** Recepción de productos de múltiples proveedores con diferentes características
2. **Gestión de Pedidos:** Comercios que pueden solicitar productos de diferentes proveedores en un mismo pedido
3. **Logística de Distribución:** Optimización de rutas con vehículos y conductores, manejando entregas exitosas e incidencias

1.2 Problemática Identificada

Del análisis del caso identificamos los siguientes retos de diseño:

- **Trazabilidad completa:** Desde que un producto entra al almacén hasta que se entrega al comercio
- **Optimización de recursos:** Asignación eficiente de vehículos y conductores
- **Manejo de incidencias:** Registro de entregas parciales o con daños
- **Control de estados:** Vehículos en mantenimiento no pueden asignarse a rutas
- **Reportes gerenciales:** Necesidad de métricas de productividad, rentabilidad y utilización

1.3 Objetivos del Diseño

Nuestro diseño busca:

1. **Integridad de datos:** Garantizar consistencia mediante restricciones y relaciones apropiadas

2. **Flexibilidad operativa:** Permitir que un comercio pida productos de múltiples proveedores
3. **Escalabilidad:** Estructura preparada para crecer en productos, zonas y vehículos
4. **Trazabilidad completa:** Cada movimiento de producto debe ser rastreable
5. **Facilidad de consulta:** Reportes eficientes mediante vistas optimizadas

2. MODELO CONCEPTUAL (ER) CON JUSTIFICACIONES

2.1 Entidades Principales Identificadas

PROVEEDOR

Justificación: Representa las empresas que suministran productos al almacén. Es fundamental para trazabilidad de origen y cálculos de rentabilidad.

Atributos clave:

- rfc: Identificador fiscal único en México
- nombre, telefono, email, direccion: Datos de contacto

Decisión: RFC como UNIQUE porque es legalmente único por empresa en México.

CATEGORÍA

Justificación: Agrupa productos por tipo (lácteos, granos, salsas, etc.). Facilita reportes de inventario por categoría y análisis de líneas de producto.

Decisión: Tabla separada (no solo un campo en PRODUCTO) porque:

- Permite agregar atributos a categorías en el futuro (ej: temperatura de almacenamiento)
- Normalización: evita repetir nombres de categorías
- Facilita consultas agrupadas

PRODUCTO

Justificación: El elemento central del negocio. Representa cada artículo que se compra, almacena y vende.

Atributos clave:

- unidad_medida: Crítico porque productos se miden en litros, kg o piezas
- precio_compra y precio_venta: Separados para calcular margen
- stock_actual: Cantidad disponible en almacén

Decisión:

- Relacionado con PROVEEDOR porque cada producto viene de un proveedor específico (según el caso: "Lácteos del Sur" provee leche y queso)
- Relacionado con CATEGORIA para clasificación

ENTRADA_ALMACEN

Justificación: Registra cada llegada de mercancía de proveedores. Es la cabecera de un "recibo de entrada".

Atributos clave:

- fecha_entrada: Timestamp automático
- usuario_registro: Auditoría de quién registró la entrada
- observaciones: Notas del almacenista

Decisión: Tabla separada del detalle porque:

- Una entrada puede tener múltiples productos (ej: Lácteos del Sur envía leche Y queso)
- Permite consultar "todas las entradas de un proveedor" sin importar productos
- Normalización 3FN

DETALLE_ENTRADA (Relación N:N)

Justificación: Conecta ENTRADA_ALMACEN con PRODUCTO. Una entrada puede tener varios productos, y un producto puede llegar en varias entradas.

Atributos:

- cantidad: Cuánto llegó de cada producto

Decisión: Tabla intermedia necesaria porque:

- Relación muchos-a-muchos
- Cada combinación entrada-producto tiene su propia cantidad
- Permite consultas como "total de leche recibida en enero"

ZONA_GEOGRAFICA

Justificación: Agrupa comercios por región (Fronteriza, Istmo, etc.). Esencial para reportes por zona.

Atributos:

- nombre_zona: Fronteriza, Istmo, Centro, etc.
- estado: Chiapas, Oaxaca, Tabasco

Decisión: Tabla separada porque:

- Permite agregar datos como distancia desde almacén central
- Facilita cálculos de cobertura geográfica
- Un comercio pertenece a UNA zona fija

COMERCIO

Justificación: Los clientes de la distribuidora. Tiendas que piden productos.

Atributos clave:

- municipio: Comitán, Juchitán, etc.
- direccion: Para planificación de rutas

Decisión: Relacionado con ZONA porque:

- Un comercio está en una zona específica
- No puede cambiar de zona frecuentemente
- Relación 1:N (una zona tiene muchos comercios)

PEDIDO

Justificación: Solicitud de productos hecha por un comercio. Cabecera del pedido.

Atributos clave:

- estado: 'Confirmado', 'En ruta', 'Entregado', 'Entregado con incidencia'
- peso_total: Calculado para validar capacidad de vehículo
- fecha_pedido: Para orden cronológico

Decisión:

- peso_total se almacena (no solo se calcula) para optimizar consultas de asignación de rutas
- estado permite tracking del ciclo de vida del pedido

DETALLE_PEDIDO (Relación N:N)

Justificación: Conecta PEDIDO con PRODUCTO. Un pedido puede tener múltiples productos (caso: Don Pepe pide leche Y maíz).

Por qué separada de DETALLE_ENTRADA:

1. **Naturaleza diferente:** Entrada = compra, Pedido = venta
2. **Propósitos distintos:** Una es para aumentar inventario, otra para disminuirlo
3. **Trazabilidad:** Permite rastrear por separado qué se compró vs qué se vendió
4. **Rentabilidad:** Facilita comparar precio_compra vs precio_venta

VEHICULO

Justificación: La flota de distribución. Cada vehículo tiene capacidad y estado operativo.

Atributos clave:

- placas: Identificador único (AB-123-CD)
- capacidad_kg: Máximo peso que puede cargar
- estado: 'Disponible', 'En ruta', 'Mantenimiento'

Decisión: estado es crítico porque:

- Previene asignar vehículos en mantenimiento
- Controla disponibilidad en tiempo real
- Requiere CHECK constraint para valores válidos

CONDUCTOR

Justificación: Empleados que manejan los vehículos.

Atributos clave:

- licencia: Número de licencia de conducir (único)
- fecha_ingreso: Para antigüedad

Decisión: Tabla separada (no parte de RUTA) porque:

- Un conductor realiza múltiples rutas
- Permite reportes de productividad por conductor
- Facilita gestión de recursos humanos

RUTA

Justificación: Plan de distribución de un día específico. Conecta vehículo, conductor y pedidos.

Atributos clave:

- fecha_ruta: Día de la distribución
- peso_total_asignado: Suma de pesos de pedidos
- estado: 'Programada', 'En curso', 'Completada'

Decisión: Requiere tanto vehículo como conductor porque:

- Ambos son recursos necesarios para ejecutar la ruta
- Permite detectar si un conductor opera múltiples vehículos
- Facilita planificación de recursos

PEDIDO_RUTA (Relación N:N)

Justificación: Un pedido puede dividirse en múltiples rutas (entrega parcial), y una ruta lleva múltiples pedidos.

Atributo adicional:

- orden_entrega: Secuencia de paradas (crítico para GPS/optimización)

Decisión: N:N porque:

- Caso real: Ruta del martes lleva pedidos de Don Pepe Y La Esquina
- Flexibilidad: Si un pedido grande no cabe en una ruta, se divide
- orden_entrega define la secuencia óptima de entregas

ENTREGA (Entidad débil)

Justificación: Registra el resultado de entregar un pedido específico.

Atributos clave:

- fecha_entrega: Cuándo se completó
- firma_digital: Confirmación del receptor
- incidencia: Texto libre (ej: "20 kg dañados")
- cantidad_dañada: Valor numérico

Por qué tabla separada (no columnas en PEDIDO):

1. **Separación de responsabilidades:** Pedido = almacén, Entrega = campo
2. **Datos opcionales:** Evita NULLs en PEDIDO para pedidos no entregados
3. **Extensibilidad:** Preparado para entregas parciales múltiples
4. **Auditoría:** Timestamp de entrega vs timestamp de pedido

Relación 1:1 con PEDIDO:

- Por ahora, un pedido tiene máximo una entrega
- Si en el futuro se permiten entregas parciales, se puede cambiar a 1:N

Cardinalidades justificadas:

Relación	Cardinalidad	Justificación
PROVEEDOR → ENTRADA	1:N	Un proveedor hace varias entregas
ENTRADA → DETALLE_ENTRADA	1:N	Una entrada tiene varios productos
PRODUCTO → DETALLE_ENTRADA	1:N	Un producto llega en varias entradas
PRODUCTO → CATEGORIA	N:1	Muchos productos de una categoría
ZONA → COMERCIO	1:N	Una zona tiene varios comercios
COMERCIO → PEDIDO	1:N	Un comercio hace varios pedidos
PEDIDO → DETALLE_PEDIDO	1:N	Un pedido tiene varios productos
PEDIDO → ENTREGA	1:1	Un pedido tiene una entrega
UTA → VEHICULO	N:1	Varias rutas usan un vehículo
UTA → CONDUCTOR	N:1	Varias rutas asignadas a un conductor

PEDIDO ↔ RUTA	N:N	Múltiples pedidos en una ruta, un pedido puede dividirse
---------------	-----	--

3. ESQUEMA RELACIONAL CON JUSTIFICACIONES

3.1 Transformación del Modelo ER a Relacional

Reglas aplicadas:

1. **Entidades fuertes** → Tablas con PK propia
2. **Relaciones 1:N** → FK en el lado N
3. **Relaciones N:N** → Tabla intermedia con PK compuesta
4. **Entidades débiles** → PK es FK de la entidad fuerte

3.2 Tablas Resultantes

Tabla 1: CATEGORIA

CREATE TABLE CATEGORIA(

id_categoria INT PRIMARY KEY AUTO_INCREMENT,

nombre_categoria VARCHAR(100) NOT NULL UNIQUE,

descripcion TEXT

);

Justificaciones:

- id_categoria: PK autoincremental para eficiencia
- nombre_categoria UNIQUE: No puede haber dos categorías con el mismo nombre
- NOT NULL: Toda categoría debe tener nombre
- descripcion opcional (TEXT) para flexibilidad

Tabla 2: PROVEEDOR

```
CREATE TABLE PROVEEDOR(  
    id_proveedor INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(200) NOT NULL,  
    rfc VARCHAR(13) NOT NULL UNIQUE,  
    telefono VARCHAR(15),  
    email VARCHAR(100),  
    direccion TEXT  
);
```

Justificaciones:

- rfc UNIQUE: RFC es identificador fiscal único en México
- VARCHAR(13): RFC mexicano tiene 12-13 caracteres
- nombre NOT NULL: Obligatorio para identificar al proveedor
- telefono, email, direccion opcionales: No todos los proveedores tienen todos los datos

Decisión de diseño: Usamos id_proveedor como PK en lugar de rfc porque:

- INT es más eficiente para JOINS que VARCHAR
- Facilita cambios si hay error en captura de RFC
- Mantiene RFC como UNIQUE para validación de negocio

Tabla 3: PRODUCTO

```
CREATE TABLE PRODUCTO(  
    id_producto INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(200) NOT NULL,  
    id_categoria INT NOT NULL,
```

```

id_proveedor INT NOT NULL,

unidad_medida VARCHAR(20) NOT NULL,

precio_compra DECIMAL(10,2) NOT NULL,

precio_venta DECIMAL(10,2) NOT NULL,

stock_actual DECIMAL(10,2) NOT NULL DEFAULT 0,

FOREIGN KEY (id_categoria) REFERENCES CATEGORIA(id_categoria),

FOREIGN KEY (id_proveedor) REFERENCES PROVEEDOR(id_proveedor)

);

```

Justificaciones:

- DECIMAL(10,2): Precisión exacta para dinero (no FLOAT por errores de redondeo)
- stock_actual DEFAULT 0: Productos nuevos inician sin inventario
- unidad_medida NOT NULL: Esencial saber si es kg, litros o piezas
- FK a CATEGORIA: Clasificación obligatoria
- FK a PROVEEDOR: Cada producto viene de un proveedor específico

Decisión: id_proveedor en PRODUCTO (no tabla intermedia) porque:

- Según el caso: "Lácteos del Sur provee leche y queso" → relación directa
- Un producto típicamente tiene un proveedor principal
- Si se necesita multi-proveedor, se puede cambiar a N:N después

Tabla 4: ENTRADA_ALMACEN

```

CREATE TABLE ENTRADA_ALMACEN(

id_entrada INT PRIMARY KEY AUTO_INCREMENT,

id_proveedor INT NOT NULL,

fecha_entrada DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

usuario_registro VARCHAR(100) NOT NULL,

observaciones TEXT,

FOREIGN KEY (id_proveedor) REFERENCES PROVEEDOR(id_proveedor)

);

```

Justificaciones:

- DATETIME DEFAULT CURRENT_TIMESTAMP: Registro automático de cuándo llegó
- usuario_registro: Auditoría (quién registró la entrada)
- observaciones opcional: No siempre hay notas especiales
- FK a PROVEEDOR: Una entrada proviene de un proveedor específico

Tabla 5: DETALLE_ENTRADA

```
CREATE TABLE DETALLE_ENTRADA(  
    id_entrada INT,  
    id_producto INT,  
    cantidad DECIMAL(10,2) NOT NULL,  
    precio_compra_historico DECIMAL(10,2) NOT NULL,  
    PRIMARY KEY (id_entrada, id_producto),  
    FOREIGN KEY (id_entrada) REFERENCES ENTRADA_ALMACEN(id_entrada) ON  
    DELETE CASCADE,  
    FOREIGN KEY (id_producto) REFERENCES PRODUCTO(id_producto)  
);
```

Justificaciones:

- PK compuesta (id_entrada, id_producto): Un producto aparece máximo una vez por entrada
- cantidad DECIMAL: Productos líquidos/fraccionables (500 litros)
- ON DELETE CASCADE: Si se borra una entrada, se borran sus detalles automáticamente
- No tiene ON DELETE CASCADECumplimiento de requisitos: 9-12 tablas (tenemos 14)
- Al menos 2 relaciones N:N (tenemos 3)
- Al menos 1 entidad débil (tenemos ENTREGA) en producto: No podemos borrar productos que han tenido movimientos

Decisión: No agregamos precio_compra aquí porque:

- Ya está en PRODUCTO
- Evita inconsistencias si cambian precios
- Si se necesita historial de precios, se crearía tabla HISTORIAL_PRECIOS

Tabla 6: ZONA_GEOGRAFICA

```
CREATE TABLE ZONA_GEOGRAFICA(  
    id_zona INT PRIMARY KEY AUTO_INCREMENT,  
    nombre_zona VARCHAR(100) NOT NULL UNIQUE,  
    estado VARCHAR(50) NOT NULL  
);
```

Justificaciones:

- nombre_zona UNIQUE: No puede haber dos zonas "Fronteriza"
- estado: Chiapas, Oaxaca, Tabasco (del caso)
- Ambos NOT NULL: Datos mínimos para definir una zona

Tabla 7: COMERCIO

```
CREATE TABLE COMERCIO(  
    id_comercio INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(200) NOT NULL,  
    id_zona INT NOT NULL,  
    municipio VARCHAR(100) NOT NULL,  
    direccion TEXT NOT NULL,  
    telefono VARCHAR(15),  
    email VARCHAR(100),  
    FOREIGN KEY (id_zona) REFERENCES ZONA_GEOGRAFICA(id_zona)  
);
```

Justificaciones:

- direccion NOT NULL: Necesaria para entregas
- municipio NOT NULL: Dato de ubicación importante
- telefono, email opcionales: No todos los pequeños comercios tienen
- FK a ZONA: Un comercio pertenece a una zona específica

Tabla 8: PEDIDO

```
CREATE TABLE PEDIDO(  
  
    id_pedido INT PRIMARY KEY AUTO_INCREMENT,  
  
    id_comercio INT NOT NULL,  
  
    fecha_pedido DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    estado VARCHAR(50) NOT NULL DEFAULT 'Confirmado',  
  
    peso_total DECIMAL(10,2),  
  
    FOREIGN KEY (id_comercio) REFERENCES COMERCIO(id_comercio)  
  
);
```

Justificaciones:

- estado DEFAULT 'Confirmado': Estado inicial al crear pedido
- peso_total nullable: Se calcula después de agregar productos
- fecha_pedido automática: Registro del momento de creación
- FK a COMERCIO: Un pedido es de un comercio específico

Decisión: peso_total se almacena (no solo se calcula) porque:

- Optimización: Evita recalcular en cada consulta
- Validación rápida: Antes de asignar a ruta, validamos vs capacidad vehículo
- Integridad: Queda congelado aunque cambien productos

Tabla 9: DETALLE_PEDIDO

```
CREATE TABLE DETALLE_PEDIDO(  
  
    id_pedido INT,  
  
    id_producto INT,  
  
    cantidad DECIMAL(10,2) NOT NULL,  
  
    precio_venta_historico DECIMAL(10,2) NOT NULL
```



```
PRIMARY KEY (id_pedido, id_producto),  
  
FOREIGN KEY (id_pedido) REFERENCES PEDIDO(id_pedido) ON DELETE CASCADE,  
  
FOREIGN KEY (id_producto) REFERENCES PRODUCTO(id_producto)  
  
);
```

Justificaciones:

- Estructura idéntica a DETALLE_ENTRADA por coherencia
- PK compuesta: Un producto máximo una vez por pedido
- ON DELETE CASCADE en pedido: Si se cancela pedido, se borran sus detalles

Tabla 10: VEHICULO

```
CREATE TABLE VEHICULO(  
  
    id_vehiculo INT PRIMARY KEY AUTO_INCREMENT,  
  
    placas VARCHAR(20) NOT NULL UNIQUE,  
  
    capacidad_kg DECIMAL(10,2) NOT NULL,  
  
    estado VARCHAR(50) NOT NULL DEFAULT 'Disponible',  
  
    modelo VARCHAR(100),  
  
    año INT  
  
);
```

Justificaciones:

- placas UNIQUE: Identificador único real del vehículo
- capacidad_kg NOT NULL: Dato crítico para asignación de rutas
- estado DEFAULT 'Disponible': Estado inicial de vehículos nuevos
- modelo, año opcionales: Datos complementarios

Decisión: Usamos id_vehiculo como PK (no placas) porque:

- INT más eficiente para FK
- Facilita actualización si hay error en placas
- Mantiene placas UNIQUE para validación

Tabla 11: CONDUCTOR

```
CREATE TABLE CONDUCTOR(  
    id_conductor INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(200) NOT NULL,  
    licencia VARCHAR(50) NOT NULL UNIQUE,  
    telefono VARCHAR(15),  
    fecha_ingreso DATE  
);
```

Justificaciones:

- licencia UNIQUE: No dos conductores con misma licencia
- fecha_ingreso tipo DATE: Solo necesitamos día, no hora
- telefono opcional: Para contacto emergencias

Tabla 12: RUTA

```
CREATE TABLE RUTA(  
    id_ruta INT PRIMARY KEY AUTO_INCREMENT,  
    fecha_ruta DATE NOT NULL,  
    id_vehiculo INT NOT NULL,  
    id_conductor INT NOT NULL,  
    estado VARCHAR(50) NOT NULL DEFAULT 'Programada',  
    peso_total_asignado DECIMAL(10,2),  
    FOREIGN KEY (id_vehiculo) REFERENCES VEHICULO(id_vehiculo),  
    FOREIGN KEY (id_conductor) REFERENCES CONDUCTOR(id_conductor)  
);
```

Justificaciones:

- fecha_ruta tipo DATE: Las rutas se planean por día
- peso_total_asignado nullable: Se calcula al agregar pedidos
- FK a vehículo Y conductor: Ambos recursos son necesarios

- estado DEFAULT 'Programada': Estado inicial

Decisión: No hay UNIQUE en (fecha_ruta, id_vehiculo) porque:

- Un vehículo podría hacer dos rutas en un día (turno AM y PM)
- Si se necesita restricción, se agrega después según reglas de negocio

Tabla 13: PEDIDO_RUTA

```
CREATE TABLE PEDIDO_RUTA(
    id_pedido INT,
    id_ruta INT,
    orden_entrega INT,
    PRIMARY KEY (id_pedido, id_ruta),
    FOREIGN KEY (id_pedido) REFERENCES PEDIDO(id_pedido),
    FOREIGN KEY (id_ruta) REFERENCES RUTA(id_ruta)
);
```

Justificaciones:

- orden_entrega: Secuencia de paradas (1=primera, 2=segunda, etc.)
- PK compuesta: Un pedido puede estar en varias rutas (entrega parcial)
- No tiene CASCADE: Borrar una ruta no debe borrar el pedido

Decisión: orden_entrega no es parte de PK porque:

- No es identificador único (puede haber orden 1 en varias rutas)
- Es solo información adicional

Tabla 14: ENTREGA

```
CREATE TABLE ENTREGA(
    id_pedido INT PRIMARY KEY,
    fecha_entrega DATETIME,
    firma_digital TEXT,
    incidencia TEXT,
    cantidad_dañada DECIMAL(10,2),
```

FOREIGN KEY (id_pedido) REFERENCES PEDIDO(id_pedido)

);

Justificaciones:

- id_pedido como PK: Relación 1:1 con PEDIDO (una entrega por pedido)
- Todos los campos nullable excepto PK: No siempre hay entrega inmediata
- firma_digital TEXT: Podría ser base64 de imagen
- incidencia y cantidad_dañada: Caso específico del queso dañado

Decisión crítica: ¿Por qué tabla separada?

1. **Ciclo de vida diferente:** Pedido se crea ≠ Entrega se registra después
2. **Responsabilidades:** Pedido = almacén, Entrega = conductor en campo
3. **Datos opcionales:** Evita NULLs en PEDIDO para pedidos pendientes
4. **Extensibilidad:** Si después se permiten entregas parciales, se cambia PK a (id_entrega)

3.3 Resumen del Esquema

Total de tablas: 14

Clasificación:

- Entidades fuertes: 8 (CATEGORIA, PROVEEDOR, PRODUCTO, ZONA, COMERCIO, VEHICULO, CONDUCTOR, ENTRADA_ALMACEN, PEDIDO, RUTA)
- Entidades débiles: 1 (ENTREGA)
- Tablas intermedias (N:N): 3 (DETALLE_ENTRADA, DETALLE_PEDIDO, PEDIDO_RUTA)

4. RESTRICCIONES IMPLEMENTADAS

4.1 Tipos de Restricciones

4.1.1 Restricciones de Unicidad (UNIQUE)

RESTRICCIÓN 1: RFC único

ALTER TABLE PROVEEDOR

ADD CONSTRAINT uq_proveedor_rfc UNIQUE (rfc);

Justificación: El RFC es identificador fiscal único por empresa en México. No pueden existir dos proveedores con el mismo RFC.

RESTRICCIÓN 2: Placas únicas

ALTER TABLE VEHICULO

ADD CONSTRAINT uq_vehiculo_placas UNIQUE (placas);

Justificación: Las placas son el identificador único real de un vehículo. No pueden existir dos vehículos con las mismas placas.

RESTRICCIÓN 3: Licencia única

ALTER TABLE CONDUCTOR

ADD CONSTRAINT uq_conductor_licencia UNIQUE (licencia);

Justificación: La licencia de conducir es personal e intransferible. No dos conductores pueden tener el mismo número de licencia.

RESTRICCIÓN 4: Nombre de categoría único

ALTER TABLE CATEGORIA

ADD CONSTRAINT uq_categoria_nombre UNIQUE (nombre_categoria);

Justificación: Evita duplicación de categorías (ej: no puede haber "Lácteos" y "lácteos" como dos categorías distintas).

RESTRICCIÓN 5: Nombre de zona único

```
ALTER TABLE ZONA_GEOGRAFICA
```

```
ADD CONSTRAINT uq_zona_nombre UNIQUE (nombre_zona);
```

Justificación: Las zonas geográficas deben ser únicas para evitar ambigüedad en reportes.

4.1.2 Restricciones de Validación (CHECK)

RESTRICCIÓN 6: Formato de RFC

```
ALTER TABLE PROVEEDOR
```

```
ADD CONSTRAINT chk_rfc_formato
```

```
CHECK (rfc REGEXP '^[A-Z]{3,4}[0-9]{6}[A-Z0-9]{3}$');
```

Justificación: El RFC mexicano tiene formato específico:

- 3-4 letras (razón social)
- 6 dígitos (fecha: AAMMDD)
- 3 caracteres alfanuméricos (homoclave)

Ejemplo válido: LDS200101ABC

RESTRICCIÓN 7: Precios positivos

```
ALTER TABLE PRODUCTO
```

```
ADD CONSTRAINT chk_precio_compra_positivo CHECK (precio_compra > 0);
```

```
ALTER TABLE PRODUCTO
```

```
ADD CONSTRAINT chk_precio_venta_positivo CHECK (precio_venta > 0);
```

Justificación: Los precios no pueden ser cero ni negativos. Todo producto debe tener un costo.

RESTRICCIÓN 8: Stock no negativo

ALTER TABLE PRODUCTO

ADD CONSTRAINT chk_stock_no_negativo CHECK (stock_actual >= 0);

Justificación: El stock puede ser cero (producto agotado), pero nunca negativo. Evita errores de captura o cálculo.

RESTRICCIÓN 9: Cantidades positivas en detalles

ALTER TABLE DETALLE_ENTRADA

ADD CONSTRAINT chk_cantidad_entrada_positiva CHECK (cantidad > 0);

ALTER TABLE DETALLE_PEDIDO

ADD CONSTRAINT chk_cantidad_pedido_positiva CHECK (cantidad > 0);

Justificación: No tiene sentido una entrada o pedido con cantidad cero o negativa. Protege contra errores de captura.

RESTRICCIÓN 10: Estados válidos para PEDIDO

ALTER TABLE PEDIDO

ADD CONSTRAINT chk_estado_pedido

CHECK (estado IN ('Confirmado', 'En ruta', 'Entregado', 'Entregado con incidencia', 'Cancelado'));

Justificación: Limita los estados posibles según el ciclo de vida del pedido definido en el caso:

- Confirmado: Pedido verificado por almacenista
- En ruta: Asignado a una ruta activa
- Entregado: Entrega exitosa
- Entregado con incidencia: Caso del queso dañado
- Cancelado: Por si se cancela antes de entregar

RESTRICCIÓN 11: Estados válidos para VEHICULO

ALTER TABLE VEHICULO

ADD CONSTRAINT chk_estado_vehiculo

CHECK (estado IN ('Disponible', 'En ruta', 'Mantenimiento', 'Fuera de servicio'));

Justificación: Del caso: "la camioneta AB-123-CD entra a mantenimiento". Estados claramente definidos:

- Disponible: Puede asignarse a rutas
- En ruta: Actualmente en operación
- Mantenimiento: NO puede asignarse (requisito del caso)
- Fuera de servicio: Accidentado o dado de baja

RESTRICCIÓN 12: Capacidad positiva

ALTER TABLE VEHICULO

ADD CONSTRAINT chk_capacidad_positiva CHECK (capacidad_kg > 0);

Justificación: Un vehículo debe tener capacidad mayor a cero para ser útil.

RESTRICCIÓN 13: Estados válidos para RUTA

ALTER TABLE RUTA

ADD CONSTRAINT chk_estado_ruta

CHECK (estado IN ('Programada', 'En curso', 'Completada', 'Cancelada'));

Justificación: Ciclo de vida de una ruta:

- Programada: Creada pero no iniciada
- En curso: Conductor en camino
- Completada: Todas las entregas realizadas
- Cancelada: Por cambios operativos

RESTRICCIÓN 14: Unidades de medida válidas

ALTER TABLE PRODUCTO

ADD CONSTRAINT chk_unidad_medida

CHECK (unidad_medida IN ('litros', 'kg', 'piezas', 'cajas'));

Justificación: Del caso: leche en litros, queso y maíz en kg. Estandariza las unidades permitidas.

4.1.3 Restricciones de Obligatoriedad (NOT NULL)

Ya incluidas en las definiciones de tablas. Ejemplos clave:

-- En PROVEEDOR

nombre VARCHAR(200) NOT NULL,

rfc VARCHAR(13) NOT NULL,

-- En PRODUCTO

nombre VARCHAR(200) NOT NULL,

precio_compra DECIMAL(10,2) NOT NULL,

precio_venta DECIMAL(10,2) NOT NULL,

unidad_medida VARCHAR(20) NOT NULL,

-- En RUTA

fecha_ruta DATE NOT NULL,

id_vehiculo INT NOT NULL,

id_conductor INT NOT NULL,

Justificación general: Los campos marcados como NOT NULL son datos mínimos indispensables para que la entidad tenga sentido operativo.

4.1.4 Restricciones de Integridad Referencial (FOREIGN KEY)

Con DELETE CASCADE:

ALTER TABLE DETALLE_ENTRADA

ADD CONSTRAINT fk_entrada_cascada

FOREIGN KEY (id_entrada) REFERENCES ENTRADA_ALMACEN(id_entrada)

ON DELETE CASCADE;

```
ALTER TABLE DETALLE_PEDIDO  
  
ADD CONSTRAINT fk_pedido_cascada  
  
FOREIGN KEY (id_pedido) REFERENCES PEDIDO(id_pedido)  
  
ON DELETE CASCADE;
```

Justificación: Si se borra una entrada o pedido (corrección de error), sus detalles deben borrarse automáticamente para evitar registros huérfanos.

Sin CASCADE (protección):

```
-- En PRODUCTO  
  
FOREIGN KEY (id_proveedor) REFERENCES PROVEEDOR(id_proveedor)  
  
-- Sin ON DELETE
```

Justificación: No podemos borrar un proveedor si tiene productos registrados. Protege la integridad histórica.

4.2 Restricciones Adicionales (Triggers necesarios)

Estas restricciones requieren lógica más compleja y se implementarán mediante triggers:

RESTRICCIÓN 15: Peso de ruta no excede capacidad de vehículo

```
CREATE TRIGGER trg_validar_capacidad_vehiculo  
  
BEFORE INSERT ON PEDIDO_RUTA  
  
FOR EACH ROW  
  
BEGIN  
  
    DECLARE v_peso_ruta DECIMAL(10,2);  
  
    DECLARE v_peso_pedido DECIMAL(10,2);  
  
    DECLARE v_capacidad DECIMAL(10,2);
```

```

-- Obtener peso actual de la ruta

SELECT COALESCE(SUM(p.peso_total), 0) INTO v_peso_ruta

FROM PEDIDO_RUTA pr

JOIN PEDIDO p ON pr.id_pedido = p.id_pedido

WHERE pr.id_ruta = NEW.id_ruta;


-- Obtener peso del pedido a agregar

SELECT peso_total INTO v_peso_pedido

FROM PEDIDO WHERE id_pedido = NEW.id_pedido;


-- Obtener capacidad del vehículo

SELECT v.capacidad_kg INTO v_capacidad

FROM RUTA r

JOIN VEHICULO v ON r.id_vehiculo = v.id_vehiculo

WHERE r.id_ruta = NEW.id_ruta;


-- Validar

IF (v_peso_ruta + v_peso_pedido) > v_capacidad THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'El peso total excede la capacidad del vehículo';

END IF;

END;

```

Justificación: Del caso: "El almacenista verifica que hay stock suficiente". Similar lógica aplica para capacidad de vehículo.

RESTRICCIÓN 16: Vehículo disponible para asignación

```

CREATE TRIGGER trg_validar_vehiculo_disponible

```

```

BEFORE INSERT ON RUTA
FOR EACH ROW
BEGIN
    DECLARE v_estado VARCHAR(50);

    SELECT estado INTO v_estado
    FROM VEHICULO
    WHERE id_vehiculo = NEW.id_vehiculo;

    IF v_estado != 'Disponible' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El vehículo no está disponible para asignación';
    END IF;
END;

```

Justificación: Del caso: "Al día siguiente, la camioneta AB-123-CD entra a mantenimiento. El coordinador intenta asignarla a otra ruta, pero no debe permitirse."

RESTRICCIÓN 17: Actualización automática de stock

```

-- Trigger para aumentar stock al registrar entrada
CREATE TRIGGER trg_aumentar_stock
AFTER INSERT ON DETALLE_ENTRADA
FOR EACH ROW
BEGIN
    UPDATE PRODUCTO
    SET stock_actual = stock_actual + NEW.cantidad
    WHERE id_producto = NEW.id_producto;
END;

```

-- Trigger para disminuir stock al confirmar pedido

CREATE TRIGGER trg_disminuir_stock

AFTER INSERT ON DETALLE_PEDIDO

FOR EACH ROW

BEGIN

UPDATE PRODUCTO

SET stock_actual = stock_actual - NEW.cantidad

WHERE id_producto = NEW.id_producto;

END;

Justificación: Del caso: "El almacenista verifica que hay stock suficiente y confirma el pedido". El stock debe actualizarse automáticamente.

4.3 Resumen de Restricciones

Tipo	Cantidad	Ejemplos
UNIQUE	5	RFC, placas, licencia, nombre_categoria, nombre_zona
CHECK	9	Formato RFC, precios positivos, estados válidos, unidades válidas
NOT NULL	20+	Incluidos en definiciones de tablas
FOREIGN KEY	13	Todas las relaciones entre tablas
Triggers	3	Capacidad, disponibilidad, stock

5. Views

Vista: Resumen de Pedidos por Zona Geográfica

```
-- 1. Resumen de pedidos por zona geográfica
CREATE VIEW vista_resumen_zonas AS
SELECT z.nombre_zona,
       COUNT(p.id_pedido) as total_pedidos,
       SUM(CASE WHEN p.estado = 'Entregado' THEN 1 ELSE 0 END) as exitosas,
       SUM(CASE WHEN p.estado = 'Entregado con incidencia' THEN 1 ELSE 0 END) as incidencias,
       (SUM(CASE WHEN p.estado = 'Entregado' THEN 1 ELSE 0 END) * 100.0 / NULLIF(COUNT(p.id_pedido),
0)) as tasa_exito
FROM ZONA_GEOGRAFICA z
LEFT JOIN COMERCIO c ON z.id_zona = c.id_zona
LEFT JOIN PEDIDO p ON c.id_comercio = p.id_comercio
GROUP BY z.nombre_zona;
```

Descripción: Provee una métrica de desempeño logístico desglosada por región (Chiapas, Oaxaca, Tabasco). Utiliza un LEFT JOIN para asegurar que todas las zonas aparezcan en el reporte y calcula la "Tasa de Éxito" comparando entregas perfectas contra el total de pedidos, permitiendo identificar áreas con problemas de logística frecuentes.

Vista: Productividad de Conductores

```
-- 2. Productividad de conductores
CREATE VIEW vista_productividad_conductores AS
SELECT co.nombre_conductor,
       COUNT(r.id_ruta) as rutas_hechas,
       COUNT(pr.id_pedido) as pedidos_totales,
       CASE
         WHEN COUNT(r.id_ruta) > 10 THEN 'Excelente'
         WHEN COUNT(r.id_ruta) BETWEEN 5 AND 10 THEN
'Normal'
         ELSE 'Baja'
       END as clasificacion
FROM CONDUCTOR co
JOIN RUTA r ON co.id_conductor = r.id_conductor
JOIN PEDIDO_RUTA pr ON r.id_ruta = pr.id_ruta
GROUP BY co.nombre_conductor;
```

Descripción: Evalúa el rendimiento del personal operativo mediante la agregación de rutas y pedidos atendidos. Implementa una estructura CASE para clasificar automáticamente a los conductores en rangos de productividad (Baja, Normal, Excelente), facilitando la toma de decisiones sobre incentivos o capacitación.

Vista: Inventario Crítico (Control de Quiebre de Stock)

```
-- 3. Inventario Crítico (Manejo de división entre cero)
CREATE VIEW vista_inventario_critico AS
SELECT p.nombre_producto,
       p.stock_actual,
       COALESCE(SUM(dp.cantidad), 0) as ventas_totales,
       COALESCE(p.stock_actual / NULLIF(SUM(dp.cantidad) / 30.0, 0), 999) as
-- 3. Inventario Crítico (Manejo de división entre cero)
LEFT JOIN DETALLE_PEDIDO dp ON p.id_producto = dp.id_producto
GROUP BY p.id_producto, p.nombre_producto, p.stock_actual;
```

Descripción: Monitorea el nivel de existencias en relación con el ritmo de ventas actual. Incorpora la función NULLIF para prevenir errores de sistema al intentar dividir entre cero en productos que no han tenido ventas recientes, asegurando que el reporte sea robusto y confiable para la planeación de compras.

Vista: Rentabilidad por Proveedor

```
-- 4. Rentabilidad por proveedor
CREATE VIEW vista_rentabilidad_proveedor AS
SELECT pr.nombre_empresa,
       SUM(dp.cantidad * dp.precio_venta_historico) as total_ventas,
       SUM(dp.cantidad * p.precio_comp_actual) as total_costo,
       (SUM(dp.cantidad * dp.precio_venta_historico) - SUM(dp.cantidad * p.precio_comp_actual)) as
       margen
FROM PROVEEDOR pr
JOIN PRODUCTO p ON pr.id_proveedor = p.id_proveedor
JOIN DETALLE_PEDIDO dp ON p.id_producto = dp.id_producto
GROUP BY pr.nombre_empresa;
```

Descripción: Realiza un balance financiero detallado comparando el costo de adquisición de los productos contra el ingreso real obtenido en las ventas. Al utilizar el **precio de venta histórico**, garantiza que el margen calculado sea exacto, incluso si los precios en el catálogo han cambiado después de la transacción.

Vista: Utilización de Flota

```
-- 5. Utilización de flota
CREATE VIEW vista_utilizacion_flota AS
SELECT v.placas,
       COUNT(r.id_ruta) as rutas_mes,
       CASE
           WHEN COUNT(r.id_ruta) > 20 THEN 'Sobrecargado'
           WHEN COUNT(r.id_ruta) BETWEEN 10 AND 20 THEN
           'Óptimo' ELSE 'Subutilizado'
       END as estado_utilizacion
FROM VEHICULO v
LEFT JOIN RUTA r ON v.id_vehiculo = r.id_vehiculo
GROUP BY v.placas;
```

Descripción: Analiza la eficiencia en el uso de los vehículos propios de la distribuidora. Cruza los datos de la flota con el historial de rutas para detectar unidades subutilizadas o vehículos que están operando por encima de su capacidad recomendada, optimizando así los ciclos de mantenimiento preventivo.

6. Queries SQL de Ejemplo

```
SELECT p.id_pedido, c.nombre_comercio, p.peso_total
FROM PEDIDO p
JOIN COMERCIO c ON p.id_comercio = c.id_comercio
WHERE p.id_pedido NOT IN (SELECT id_pedido FROM
PEDIDO_RUTA);
```

```
SELECT r.id_ruta, v.capacidad_kg, SUM(p.peso_total) as peso_asignado
FROM RUTA r
JOIN VEHICULO v ON r.id_vehiculo = v.id_vehiculo
JOIN PEDIDO_RUTA pr ON r.id_ruta = pr.id_ruta
JOIN PEDIDO p ON pr.id_pedido = p.id_pedido
GROUP BY r.id_ruta, v.capacidad_kg
HAVING SUM(p.peso_total) > v.capacidad_kg * 0.9; -- Alerta de
sobrecarga
```

7. Preguntas

1. ¿Por qué escogieron el Caso 3-A?

Elegimos este caso porque representaba el desafío técnico más completo y cercano a un entorno empresarial real para un equipo de tres integrantes y nos pareció el más atractivo.

- **Complejidad Arquitectónica:** El caso requiere gestionar tres flujos de datos distintos y entrelazados: el flujo de suministro (proveedores), el flujo comercial (ventas/pedidos) y el flujo operativo (logística/rutas).
- **Desafío de Integridad:** Nos permitía implementar una lógica de estados avanzada, como el manejo de vehículos en mantenimiento que bloquean la asignación de rutas, lo cual es un excelente caso de uso para triggers y restricciones de negocio.

2. ¿Por qué lo diseñaron así?

El diseño se enfocó en la mayor preservación posible de la información.

A. Preservación de Datos Históricos

Decidimos incluir los precios unitarios directamente en las tablas de detalle (DETALLE_PEDIDO y DETALLE_ENTRADA).

Justificación: Si el precio de la leche sube mañana en la tabla de PRODUCTO, los pedidos realizados hoy deben mantener su valor original. Diseñarlo de otra forma corrompería la contabilidad histórica del negocio.

B. Uso de Entidades Débiles para Incidencias

Modelamos la ENTREGA como una entidad débil ligada al PEDIDO.

- **Separación de Responsabilidades:** El PEDIDO nace en la oficina/almacén, mientras que la ENTREGA ocurre en el campo.
- **Manejo de Incidencias:** Esto permite que el sistema registre entregas parciales o daños (como los 20kg de queso dañados) sin necesidad de modificar el pedido original, manteniendo la trazabilidad de qué salió vs. qué se recibió.

C. Automatización mediante Triggers

En lugar de depender únicamente de la aplicación, delegamos la integridad al motor de la base de datos mediante triggers para la actualización de stock y validación de flota.

- **Seguridad:** Garantizamos que el stock siempre sea consistente con las entradas y salidas reales.
- **Reglas de Negocio:** El sistema prohíbe físicamente asignar un vehículo en mantenimiento a una ruta, asegurando que los datos reflejen la realidad operativa.

D. Normalización y Escalabilidad

Separamos las categorías y zonas en tablas independientes.

- **Normalización:** Evitamos redundancia de datos y facilitamos el crecimiento (ej. si mañana la distribuidora abre una nueva región o añade tipos de almacenamiento específicos por categoría).